

Job e CronJob

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere la differenza tra un **Job** e un workload continuo come un **Deployment**
 - Configurare un **Job** con **completions**, **parallelism** e **backoffLimit** per task batch semplici e paralleli
 - Spiegare la differenza tra **completionMode: NonIndexed** e **Indexed** e i casi d'uso di ciascuno
 - Creare un **CronJob** con sintassi cron, **concurrencyPolicy** e policy di retention degli eseguiti
 - Avviare manualmente un'esecuzione da un **CronJob** con **oc create job**
 - Applicare le best practice ARO per Job batch che interagiscono con servizi Azure
-

Teoria e Concetti Chiave

Cos'è un Job

Un **Job** crea uno o più Pod e garantisce che un numero specificato di essi **completi con successo** (exit code 0). A differenza di un **Deployment**, il cui obiettivo è mantenere Pod sempre in esecuzione, un Job termina una volta raggiunto il numero di completamenti desiderati.

```
Deployment → vuole Pod sempre Running (replica continua)
Job         → vuole Pod che completano con successo (poi finisce)
```

Casi d'uso tipici:

- Migrazione dati / schema DB
- Generazione di report periodici
- Backup di database
- Elaborazione batch di file
- Operazioni di pulizia (cleanup) di risorse
- Invio di email/notifiche in batch

Parametri principali di un Job

Campo	Default	Descrizione
<code>spec.completions</code>	1	Numero totale di completamenti richiesti prima che il Job sia considerato completato
<code>spec.parallelism</code>	1	Numero massimo di Pod che possono girare contemporaneamente
<code>spec.backoffLimit</code>	6	Numero di tentativi prima che il Job venga considerato fallito
<code>spec.activeDeadlineSeconds</code>	nessuno	Timeout totale del Job in secondi (include tutti i tentativi)
<code>spec.ttlSecondsAfterFinished</code>	nessuno	Secondi dopo i quali il Job completato viene eliminato automaticamente

completionMode: NonIndexed vs Indexed

Modalità	Comportamento	Caso d'uso
NonIndexed (default)	Tutti i Pod sono equivalenti e intercambiabili; il Job è completato quando <code>completions</code> Pod hanno completato	Task batch omogenei (es. invia N email)
Indexed	Ogni Pod ottiene un indice univoco (0 a N-1) disponibile in <code>JOB_COMPLETION_INDEX</code> env; ogni indice deve completare	Task batch con partizionamento dell'input (es. elabora file[0], file[1], file[2]...)

Con `completionMode: Indexed`, l'hostname del Pod include l'indice:

```
job-name-0 → JOB_COMPLETION_INDEX=0
job-name-1 → JOB_COMPLETION_INDEX=1
job-name-2 → JOB_COMPLETION_INDEX=2
```

backoffLimit e gestione dei fallimenti

Se un Pod del Job fallisce (exit code $\neq 0$), Kubernetes lo riavvia (o ne crea uno nuovo). Dopo `backoffLimit` fallimenti, il Job viene marcato come **Failed** e non vengono creati altri Pod.

```
backoffLimit: 3 → dopo 3 fallimenti totali: Job → Failed
```

⚠ Attenzione: Il backoff cresce esponenzialmente (10s, 20s, 40s, 80s...) tra un tentativo e il successivo per evitare di sovraccaricare il sistema.

startingDeadlineSeconds

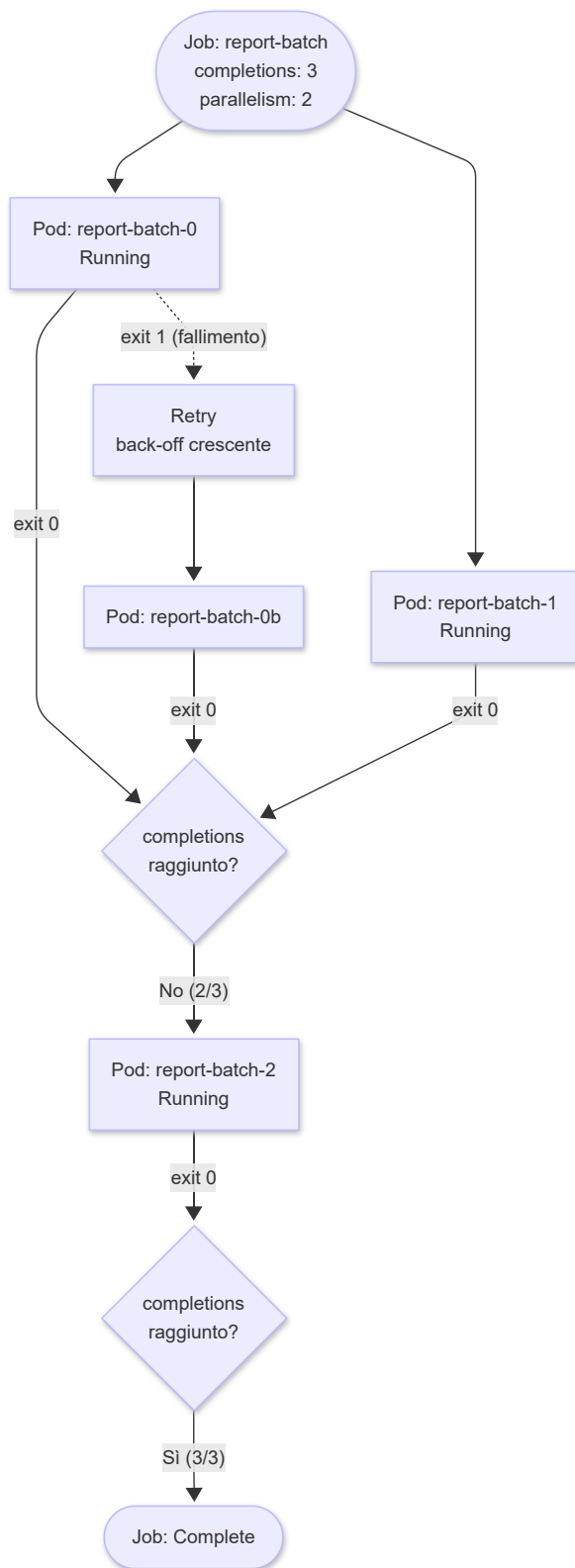
Se il cluster è irraggiungibile o il CronJob non riesce a partire in tempo (es. per mancanza di risorse), **startingDeadlineSeconds** definisce dopo quanti secondi dall'orario pianificato l'esecuzione viene considerata "saltata":

```
startingDeadlineSeconds: 300 # se non parte entro 5 minuti dall'orario, salta
```

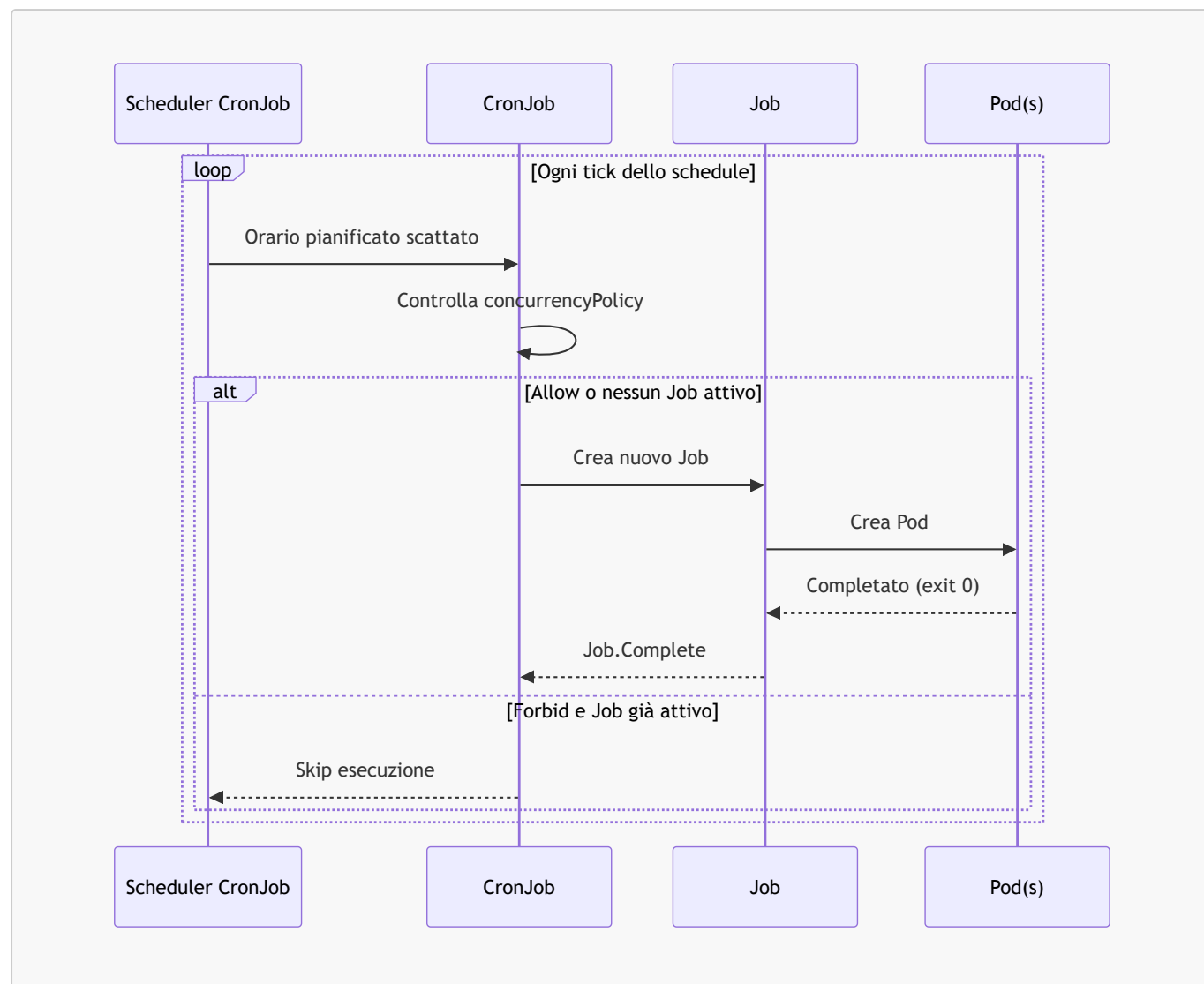
⚠ Attenzione: Se **startingDeadlineSeconds** non è impostato e il cluster è down per più di 100 orari pianificati consecutivi, il CronJob viene disabilitato automaticamente. Imposta sempre **startingDeadlineSeconds** per CronJob critici.

Diagrammi

Flusso di esecuzione di un Job



Ciclo di vita di un CronJob



Comandi Pratici con Output Atteso

```
# Creare un Job da riga di comando (immagine di test)
oc create job hello --image=registry.access.redhat.com/ubi9/ubi-minimal \
  -- /bin/bash -c "echo 'Hello from Job'; sleep 5; exit 0"
```

```
# Output atteso:
# job.batch/hello created
```

```
# Elencare i Job nel progetto corrente
oc get jobs
```

```
# Output atteso:
# NAME      COMPLETIONS  DURATION  AGE
# hello     1/1          8s        30s
```

```
# Seguire i log del Pod creato dal Job
oc logs -l job-name=hello
```

```
# Output atteso:
# Hello from Job
```

```
# Mostrare i dettagli di un Job inclusi tentativi e completamenti
oc describe job hello
```

```
# Output atteso (estratto):
# Name:          hello
# Namespace:     myproject
# Completions:   1
# Duration:      8s
# Pods Statuses: 0 Active (0 Ready) / 1 Succeeded / 0 Failed
# Events:
#   Normal SuccessfulCreate 40s   job-controller Created pod: hello-xk2lp
#   Normal Completed       32s   job-controller Job completed
```

```
# Creare un CronJob che esegue ogni minuto
oc create cronjob minutely-hello \
  --image=registry.access.redhat.com/ubi9/ubi-minimal \
  --schedule="*/1 * * * *" \
  -- /bin/bash -c "date && echo 'CronJob run'"
```

```
# Output atteso:
# cronjob.batch/minutely-hello created
```

```
# Elencare i CronJob
oc get cronjobs
```

```
# Output atteso:
# NAME          SCHEDULE          SUSPEND   ACTIVE   LAST SCHEDULE   AGE
# minutely-hello */1 * * * *      False     0        25s       3m
```

```
# Avviare manualmente un'esecuzione da un CronJob esistente (utile per test/debug)
oc create job --from=cronjob/minutely-hello manual-run-001
```

```
# Output atteso:
# job.batch/manual-run-001 created
```

```
# Suspendere un CronJob (non elimina i Job già in esecuzione)
oc patch cronjob minutely-hello -p '{"spec":{"suspend":true}}'
```

```
# Output atteso:
# cronjob.batch/minutely-hello patched
```

```
# Visualizzare la history dei Job creati da un CronJob
oc get jobs -l job-name=minutely-hello
```

```
# Alternativa: elencare tutti i Job con label del CronJob
oc get jobs --sort-by=.metadata.creationTimestamp
```

```
# Eliminare tutti i Job completati di un CronJob
oc delete jobs -l job-name=minutely-hello --field-selector=status.conditions[0].type=Complete
```

Esempi YAML Completi

Job con parallelismo e backoffLimit

```
apiVersion: batch/v1
kind: Job
metadata:
  name: data-migration
  namespace: myproject
  labels:
    app: data-migration
    version: "v1.2"
spec:
  # Numero totale di completamenti richiesti
  completions: 6
  # Massimo 3 Pod in esecuzione contemporaneamente
  parallelism: 3
  # Massimo 2 fallimenti prima di dichiarare il Job Failed
  backoffLimit: 2
  # Timeout assoluto del Job: se non completa in 10 minuti, viene terminato
  activeDeadlineSeconds: 600
  # Elimina automaticamente il Job 1 ora dopo il completamento
  ttlSecondsAfterFinished: 3600
  template:
    metadata:
      labels:
        app: data-migration
    spec:
      # OnFailure: riavvia il container (non crea un nuovo Pod) al fallimento
      restartPolicy: OnFailure
      containers:
        - name: migrator
          image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
          command: ["/bin/bash", "-c"]
          args:
            - |
              echo "Migrating batch $JOB_COMPLETION_INDEX..."
              # ... logica di migrazione dati ...
              echo "Batch complete"
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "500m"
          env:
            - name: DB_HOST
              valueFrom:
                configMapKeyRef:
                  name: app-config
```

```

        key: db-host
      - name: DB_PASSWORD
        valueFrom:
          secretKeyRef:
            name: db-secret
            key: password

```

Job con completionMode Indexed

```

apiVersion: batch/v1
kind: Job
metadata:
  name: file-processor
  namespace: myproject
spec:
  # 5 file da elaborare (indici 0..4)
  completions: 5
  parallelism: 2
  # Ogni Pod riceve un indice univoco (0-4)
  completionMode: Indexed
  backoffLimit: 3
  template:
    spec:
      restartPolicy: OnFailure
      containers:
      - name: processor
        image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
        command: ["/bin/bash", "-c"]
        args:
          # JOB_COMPLETION_INDEX è iniettato automaticamente da Kubernetes
          - "echo Processing file number $JOB_COMPLETION_INDEX"
        resources:
          requests:
            memory: "64Mi"
            cpu: "50m"
          limits:
            memory: "128Mi"
            cpu: "200m"

```

CronJob per backup notturno

```

apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
  namespace: myproject
  labels:
    app: nightly-backup
spec:

```

```

# Ogni giorno alle 02:00 UTC
schedule: "0 2 * * *"
# Se un backup è già in esecuzione, salta il successivo
concurrencyPolicy: Forbid
# Se non parte entro 10 minuti dall'orario, salta
startingDeadlineSeconds: 600
# Mantieni gli ultimi 3 backup completati e gli ultimi 3 falliti
successfulJobsHistoryLimit: 3
failedJobsHistoryLimit: 3
# Non sospeso (true = pausa temporanea del CronJob)
suspend: false
jobTemplate:
  spec:
    # Timeout massimo del backup: 2 ore
    activeDeadlineSeconds: 7200
    backoffLimit: 1
    ttlSecondsAfterFinished: 86400 # elimina dopo 24h
    template:
      metadata:
        labels:
          app: nightly-backup
      spec:
        restartPolicy: OnFailure
        containers:
          - name: backup
            image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
            command: ["/bin/bash", "-c"]
            args:
              - |
                TIMESTAMP=$(date +%Y%m%d_%H%M%S)
                echo "Starting backup at $TIMESTAMP"
                # Logica di backup: dump DB, upload su Azure Blob, ecc.
                echo "Backup completed successfully"
        resources:
          requests:
            memory: "256Mi"
            cpu: "200m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        env:
          - name: BACKUP_STORAGE_ACCOUNT
            valueFrom:
              secretKeyRef:
                name: azure-storage-secret
                key: accountName
          - name: BACKUP_STORAGE_KEY
            valueFrom:
              secretKeyRef:
                name: azure-storage-secret
                key: accountKey

```

Note Specifiche per Azure ARO

Job con Accesso ad Azure Storage

I Job batch su ARO spesso necessitano di leggere/scrivere su Azure Blob Storage. Il modo raccomandato è usare Azure Workload Identity (Managed Identity):

```
# Verificare se Workload Identity è abilitato sul cluster ARO
oc get authentication cluster -o yaml | grep -i workload

# Creare un ServiceAccount con annotazione Azure Identity
oc create serviceaccount backup-sa -n myproject
oc annotate serviceaccount backup-sa \
  azure.workload.identity/client-id=<client-id> \
  -n myproject
```

 **Suggerimento:** Non inserire mai credenziali Azure in variabili d'ambiente in chiaro nel Job YAML. Usa sempre Secrets di Kubernetes o Azure Workload Identity per l'accesso ad Azure Storage, Key Vault e altri servizi.

Timezone nei CronJob (OCP 4.14+)

A partire da OCP 4.14 (Kubernetes 1.27+), il campo `timeZone` è supportato nativamente nei CronJob:

```
spec:
  schedule: "0 8 * * 1-5"
  timeZone: "Europe/Rome" # Supportato da OCP 4.14+
# Senza timeZone, lo schedule usa UTC
```

 **Attenzione:** Prima di OCP 4.14, per schedulare in un fuso orario specifico era necessario adattare manualmente l'orario UTC nel campo `schedule`.

Resource Quotas e Job su ARO

Se il namespace ha una **ResourceQuota**, i Job devono rispettare i limiti configurati. Un Job che richiede risorse superiori alla quota disponibile rimarrà bloccato con Pod in **Pending**:

```
# Verificare le ResourceQuota del namespace
oc describe resourcequota -n myproject
```

```
# Output atteso (esempio):
```

```
# Name:                core-quota
# Resource              Used      Hard
# -----
# pods                  8          20
# requests.cpu          1200m      4000m
# requests.memory       1280Mi     8Gi
# limits.cpu            3200m      16000m
# limits.memory         3840Mi     32Gi
```

Riepilogo dei Punti Chiave

- Un **Job** crea Pod che devono completare con successo; un **Deployment** crea Pod che devono sempre essere in esecuzione
 - **completions** definisce quanti Pod devono completare; **parallelism** quanti possono girare simultaneamente
 - **backoffLimit** limita il numero di tentativi; **activeDeadlineSeconds** impone un timeout assoluto
 - **completionMode: Indexed** assegna un indice univoco a ogni Pod tramite la variabile **JOB_COMPLETION_INDEX**
 - Un **CronJob** crea automaticamente Job secondo uno schedule cron; **concurrencyPolicy** controlla il comportamento in caso di sovrapposizione
 - **startingDeadlineSeconds** previene la creazione di Job "in ritardo"; impostarlo sempre su CronJob critici
 - Su OCP 4.14+ usare il campo **timeZone** nei CronJob per evitare conversioni manuali da fuso locale a UTC
-

Domande di Verifica

1. Un Job ha **completions: 4** e **parallelism: 2**. Quanti Pod girano contemporaneamente al massimo?

- A) 4
- B) 1
- C) 2 ☒
- D) Dipende dalla **restartPolicy**

2. Cosa succede a un CronJob con **concurrencyPolicy: Forbid** quando lo schedule scatta ma un Job è ancora in esecuzione?

- A) Il Job in esecuzione viene eliminato e ne viene avviato uno nuovo
- B) Il nuovo Job viene accodato e parte non appena il precedente termina
- C) La nuova esecuzione viene saltata ☒
- D) Entrambi i Job vengono eseguiti in parallelo

3. Quale **restartPolicy** è obbligatoria per i Pod creati da un Job?

- A) **Always**
- B) **OnFailure** o **Never** ☒
- C) Qualsiasi valore — non ci sono restrizioni
- D) **Never** è l'unico valore supportato

4. Cosa fornisce **completionMode: Indexed** a ogni Pod del Job?

- A) Un nome DNS stabile
- B) Un volume PVC dedicato
- C) Un indice univoco (0 a N-1) disponibile in **JOB_COMPLETION_INDEX** ☒
- D) Un nodo dedicato tramite affinity

5. Un CronJob critico deve girare ogni giorno alle 03:00 ora italiana. Il supporto **timeZone** non è disponibile (OCP < 4.14). Quale schedule usare in estate (CEST = UTC+2)?

- A) `0 3 * * *`
- B) `0 1 * * *` ☒ (03:00 CEST = 01:00 UTC)
- C) `0 5 * * *`
- D) `0 2 * * *`